

Design Decision: Why Script UTXO Certificate Storage Was Replaced by a Minting Policy Approach

1. Initial Approach

The first implementation used a `certificate_validator` spending script.

Each certificate was planned to be stored as an independent UTXO at a certificate validator address:

```
Certificate Validator Address
├─ UTXO 1 → CertificateDatum(GEM_001)
├─ UTXO 2 → CertificateDatum(GEM_002)
├─ UTXO 3 → CertificateDatum(GEM_003)
```

Each UTXO carried an inline datum:

```
CertificateDatum
├─ issuer_id
├─ report_id
├─ report_type
├─ gem_id
├─ cert_hash
├─ issued_at
├─ schema_version
├─ document_cid
└─ issuer_pkh
```

The validator checked:

- report_type is between 1 and 8
- cert_hash length is 32 bytes
- issuer_pkh signed the transaction

At first, this seemed suitable because each certificate could be represented as an immutable on-chain record.

2. Why This Method Fails for Certificate Issuance Authorization

The main issue is that a spending validator only runs when a UTXO is spent.

It does not run when someone creates a UTXO at the script address.

This means a user could create a certificate UTXO with any datum, even if they are not an authorized issuer.

For example:

```
User creates UTXO at certificate script address
└ datum says issuer_pkh = fake issuer
```

The validator does not check this at creation time.

It only checks later if that UTXO is spent.

Therefore, the script UTXO method does not properly enforce:

```
Only authorized issuers can issue certificates.
```

It only enforces rules when the certificate UTXO is consumed later.

3. Why Off-chain Checking Is Not Enough

One temporary workaround is to check issuer authorization in the frontend or backend before creating the certificate UTXO.

However, this is not trustless.

A malicious user could bypass the frontend/backend and directly submit a transaction to the blockchain.

Since the certificate validator does not execute during UTXO creation, the blockchain itself would not reject the unauthorized certificate UTXO.

This weakens the security model.

The goal of the system is not only to store certificate data, but also to enforce issuer authorization on-chain.

4. Scalability and Indexing Problems

Using script UTXOs for every certificate also creates practical issues.

Each certificate becomes a separate UTXO locked at a script address.

This is valid in Cardano, but querying and managing many certificate UTXOs requires an external indexer.

The blockchain itself does not provide easy application-level search such as:

```
Find all certificates for GEM_001
Find all certificates issued by GIA
Find latest certificate for report R001
```

These queries still require an indexer or backend database.

Therefore, script UTXOs do not remove the need for off-chain indexing.

5. Why a Minting Policy Is a Better Fit

A minting policy runs exactly when a token is minted or burned.

That makes it the correct place to enforce certificate issuance rules.

The certificate minting policy can check:

1. The issuer signed the transaction
2. The issuer is listed in the registry UTXO
3. The registry UTXO is included as a reference input
4. Exactly one certificate token is minted

This directly enforces:

```
Only authorized issuers can create certificate tokens.
```

Unlike the spending validator, the minting policy executes at the exact moment of certificate creation.

6. Final Chosen Architecture

The improved architecture separates responsibilities:

```
Issuer Registry Validator
  → stores authorized issuer list as a registry UTXO

Certificate Minting Policy
  → controls certificate token creation

Certificate Token / Metadata
  → represents the issued certificate

IPFS
  → stores certificate documents and images

Indexer / Backend
  → enables searching by gemId, issuer, and reportId
```

7. Why the Registry Still Works

The registry does not store certificates.

It only stores authorized issuer public key hashes:

```
RegistryDatum
├─ admin
├─ issuers[]
└─ version
```

This keeps the registry small and scalable.

Even if the system has thousands of certificates, the registry only changes when issuer permissions change.

8. Final Design Justification

The script UTXO certificate approach was rejected because it validates certificate records only when they are spent, not when they are created.

For a certificate issuance system, the most important requirement is to control who can create certificates.

A minting policy is better suited because it validates the transaction at the time of token creation.

Therefore, the project adopts a minting-policy-based certificate issuance model, while keeping the issuer registry as a stateful UTXO-based access-control mechanism.

9. Summary

Approach	Problem
Certificate script UTXO	Validator does not run during UTXO creation
Frontend/backend authorization only	Can be bypassed
Many certificate UTXOs	Still requires indexing
Minting policy	Enforces authorization at certificate creation

Final decision:

```
Use registry UTXO for issuer authorization.  
Use minting policy for certificate issuance.  
Use tokens/metadata/IPFS for certificate representation.
```